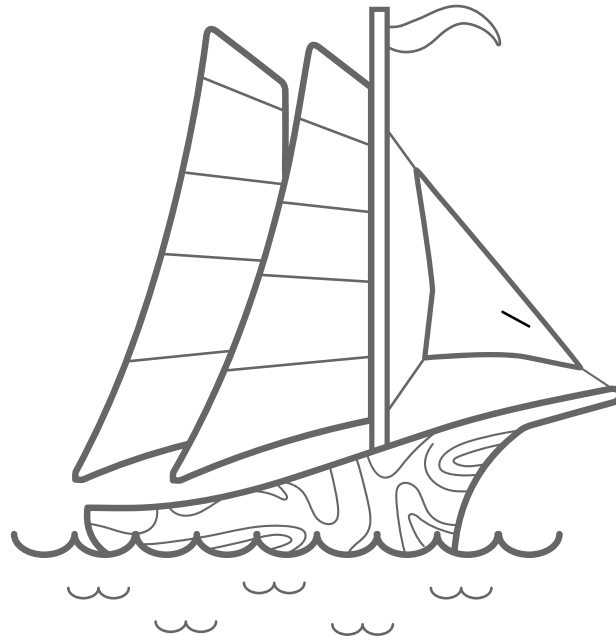


LangQuest

Benutzerhandbuch

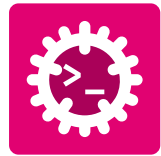


LangQuest

HES-SO Valais//Wallis
Systems Engineering • Infotronics
Advanced Programming • Silvan Zahno
Fall Semester 2026 • I3

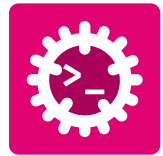
Silvan Zahno, Amand Axel

24.06.2026 • v1.0.0 • final



Inhalt

1 Einführung	3
2 Installation	4
2.1 Rust-Toolchain	4
2.1.1 Installation	4
2.1.2 cargo -Erweiterungen installieren	5
2.1.3 Überprüfung	6
2.2 LangQuest	7
2.2.1 Installation	7
2.2.2 Schnelleinstieg	7
2.2.3 Dateitypen dem Editor zuordnen	7
2.3 Zed-Code-Editor	9
2.3.1 Installation	9
2.3.2 Überprüfung	9
2.4 Just - Kommando-Runner	10
2.4.1 Installation	10
2.4.2 Überprüfung	10
3 LangQuest Anleitung	11
3.1 LangQuest starten	11
3.2 Übersichtsseite	12
3.2.1 Kurzbefehle im Überblick	13
3.3 Übungsseite	14
3.3.1 Theorie	14
3.3.2 Aufgaben	14
3.3.3 Debug	14
3.3.4 Ausgabe	15
3.3.5 Tipps und Lösung	15
3.3.6 Kurzbefehle im Überblick	16
3.4 Info-Seite	17
3.5 Fehlerbehebung	17
Glossary	19



1 Einführung

LangQuest wie auf der GitHub-Seite beschrieben:



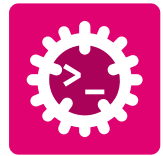
LangQuest

A terminal-based, interactive programming exercise runner. Inspired by Rustlings and 100 Exercises to Learn Rust, LangQuest extends the concept to multiple languages - work through hands-on exercises in Rust, Go, C++, Python, RISC-V assembly, and Markdown with real-time feedback, progress tracking, and a built-in hint system.

Dieses Tool wurde von **Zahno Silvan** entwickelt und unter der MIT-Lizenz veröffentlicht. Der Quellcode ist unter <https://github.com/tschinz/langquest> verfügbar.

LangQuest ist ein interaktives Tool, dessen Schnittstelle dank **Ratatui** in der Befehlszeile ausgeführt wird und es Ihnen ermöglicht, praktische Übungen in Rust, Go, C++, Python, RISC-V-Assembly und Markdown mit Echtzeit-Feedback, Fortschrittsverfolgung und einem integrierten Hinweis-System zu bearbeiten.

Das folgende Dokument bietet eine umfassende Benutzeranleitung für LangQuest und behandelt Installation, Nutzung und Fehlerbehebung, damit Sie dieses interaktive Lernwerkzeug optimal nutzen können.



2 | Installation

Dieser Abschnitt behandelt die Installation aller Werkzeuge, die mit LangQuest verbunden sind. Arbeiten Sie die Unterabschnitte in Reihenfolge durch - einige Tools bauen auf vorherigen Installationen auf.

2.1 Rust-Toolchain



Figure 1 - Rust Programming Language

LangQuest wird über **Cargo**, den Paketmanager von Rust, installiert.

Rust wird über **rustup**, den offiziellen Installer der Rust-Toolchain, installiert und verwaltet. **rustup** installiert den Compiler (**rustc**), das Build- und Paketwerkzeug (**cargo**) sowie zusätzliche Ziele und Komponenten.

2.1.1 Installation

Vollständige Anweisungen finden Sie unter rust-lang.org/tools/install.

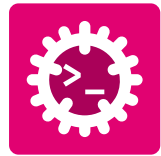
Öffnen Sie ein Terminal und führen Sie aus:

```
1 curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```



Folgen Sie den Anweisungen am Bildschirm (die Standardinstallation wird empfohlen). Öffnen Sie danach das Terminal neu oder laden Sie die Umgebung:

```
1 source "$HOME/.cargo/env"
```



Laden Sie **rustup-init.exe** von <https://win.rustup.rs/> herunter und starten Sie es.

Falls **MSVC**-Tools nicht installiert sind, bestätigen Sie die Installation auf Nachfrage:



Fahren Sie dann mit der Installation fort (die Standardinstallation wird empfohlen):

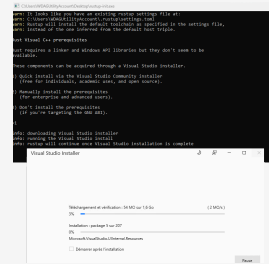


Figure 2 - MSVC installation through rustup

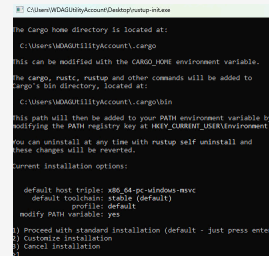
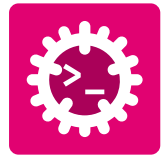


Figure 3 - Rust installation through rustup on Windows

2.1.2 cargo-Erweiterungen installieren

Führen Sie in einem neuen Terminal die folgenden Befehle aus, um nützliche **cargo**-Erweiterungen für Formatierung und Linting zu installieren:

```
1 # Install rustfmt for code formatting
2 rustup component add rustfmt
3 # Install clippy for linting and code analysis
4 rustup component add clippy
```



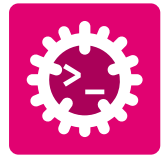
2.1.3 Überprüfung

Prüfen Sie, dass die folgenden Befehle eine Versionsnummer ohne Fehler ausgeben:



```
1 rustup --version
2 rustc --version
3 cargo --version
4 cargo fmt --version
5 cargo clippy --version
```

Alle Befehle sollten eine Versionsnummer ausgeben. Falls nicht, stellen Sie sicher, dass **\$HOME/.cargo/bin** (macOS/Linux) bzw. **%USERPROFILE%\.cargo\bin** (Windows) im **PATH** enthalten ist.



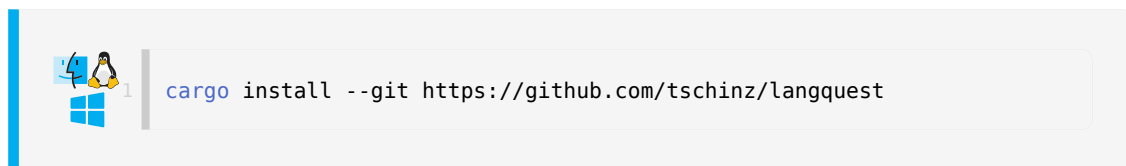
2.2 LangQuest



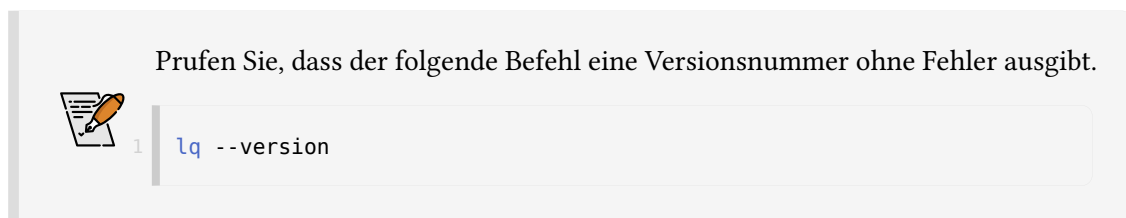
Figure 4 - LangQuest - vocabulary quiz tool

LangQuest ist ein terminalbasiertes Vokabel-Quiz-Tool zum Üben von Programmierbegriffen und -konzepten. Es wird als binäre Rust-Crate verteilt und über **cargo** installiert.

2.2.1 Installation



```
1 cargo install --git https://github.com/tschinz/langquest
```

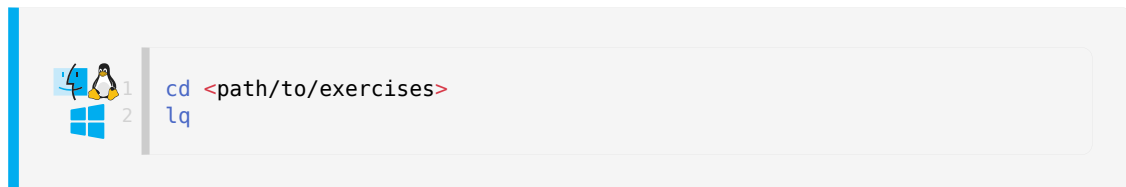


Prüfen Sie, dass der folgende Befehl eine Versionsnummer ohne Fehler ausgibt.

```
1 lq --version
```

2.2.2 Schnelleinstieg

Zum Start wechseln Sie in einen Ordner mit LangQuest-Ubungen und führen aus:



```
1 cd <path/to/exercises>
2 lq
```

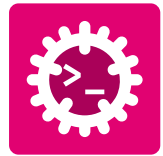
Das Programm erkennt Struktur und Inhalte der Übungen automatisch.

2.2.3 Dateitypen dem Editor zuordnen

LangQuest öffnet Übungsdateien im Standardeditor für:

- **Rust:** *.rs
- **Python:** *.py
- **Go:** *.go
- **C++:** *.c, *.h, *.cpp, *.hpp
- **RISC-V:** *.s, *.asm
- **Markdown:** *.md

Um die Nutzung zu vereinfachen, ordnen Sie die in LangQuest verwendeten Dateieindungen Ihrem bevorzugten Editor zu:



Rechtsklick auf eine Datei mit passender Endung (z. B. **exercise.rs**) -> **Informationen** -> **Offnen mit** -> Editor wahlen (z. B. Zed) -> **Alle andern...** -> Bestatigen -> Schliessen

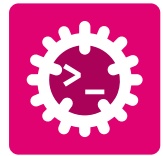


Abhangig vom Betriebssystem

Z. B. Ubuntu-GUI: **Einstellungen** -> **Standardanwendungen** -> **Standardanwendungen nach Dateityp festlegen** -> Endungen suchen (z. B. **.rs**) -> Editor auswählen (z. B. VSCode oder Zed) -> Schliessen



Rechtsklick auf eine Datei mit passender Endung (z. B. **exercise.rs**) -> **Offnen mit** -> **Andere App auswählen** -> Editor auswählen (z. B. Zed) -> **Immer diese App zum Offnen von .xx-Dateien verwenden** aktivieren -> **OK**



2.3 Zed-Code-Editor



Figure 5 - Zed - high-performance code editor

Zed ist ein leistungsstarker, kollaborativer Code-Editor auf Rust-Basis. Er ist der primäre Editor in diesem Guide.

2.3.1 Installation

Besuchen Sie zed.dev und laden Sie den Installer für Ihre Plattform herunter.



Laden Sie die **.dmg** herunter, öffnen Sie sie und ziehen Sie *Zed* in den Ordner *Programme*.

Installieren Sie das Zed-CLI in Zed: Drücken Sie **cmd-shift-p** und geben Sie **cli:install cli binary** ein



Öffnen Sie ein Terminal und führen Sie das offizielle Installationsskript aus:

```
curl https://zed.dev/install.sh | sh
```



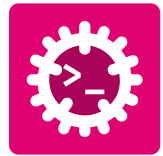
Laden Sie den **.exe**-Installer herunter und führen Sie ihn aus.

Fügen Sie **zed.exe** zu Ihrem **PATH** hinzu, falls dies nicht automatisch geschieht.

2.3.2 Überprüfung



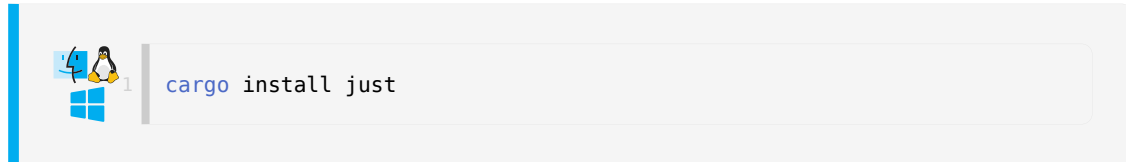
Öffnen Sie Zed und bestätigen Sie, dass der Startbildschirm erscheint. Melden Sie sich mit Ihrem GitHub-Konto an, um **Artificial Intelligence (AI)**-Funktionen zu aktivieren.



2.4 Just - Kommando-Runner

just ist ein praktischer Command-Runner (ähnlich wie **make**), der ein **justfile** im Projekt liest. Er wird als binäre Rust-Crate verteilt und über **cargo** installiert.

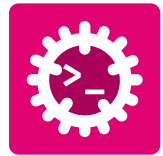
2.4.1 Installation



Die Erstinstallation kann einige Minuten dauern, da Crate und Abhängigkeiten heruntergeladen und kompiliert werden.

2.4.2 Überprüfung





3 | LangQuest Anleitung

3.1 LangQuest starten

LangQuest verlangt keinen bestimmten Speicherort für Übungen, aber die Verzeichnisstruktur muss einem festen Layout folgen:

```
my-exercises/
├── lq.toml                ← auto-created by lq on first run
├── 01-basics/             ← module (prefixed with NN-)
│   ├── 01-hello-world/   ← exercise (prefixed with NN-)
│   │   ├── 01-theory.md
│   │   ├── 02-task.md
│   │   ├── main.rs
│   │   └── solution/
│   │       ├── main.rs
│   │       └── solution.md
│   └── 02-variables/
│       └── ...
└── 02-control-flow/
    └── ...
```

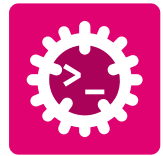
Speichern Sie die Übungen Ihres Kurses in einem eigenen Ordner.

Wechseln Sie im Terminal in den Ordner mit den Übungen und starten Sie LangQuest mit **lq**:

```
cd <path/to/exercises>
lq
```



Es wird empfohlen, LangQuest in einem in **Zed** geöffneten Terminal zu starten, damit die Übungsdateien direkt im selben Editor geöffnet werden. Die Screenshots in diesem Leitfaden stammen aus Zed.



3.2 Übersichtsseite

Öffnen Sie den Ordner mit den Übungen in **Zed**:

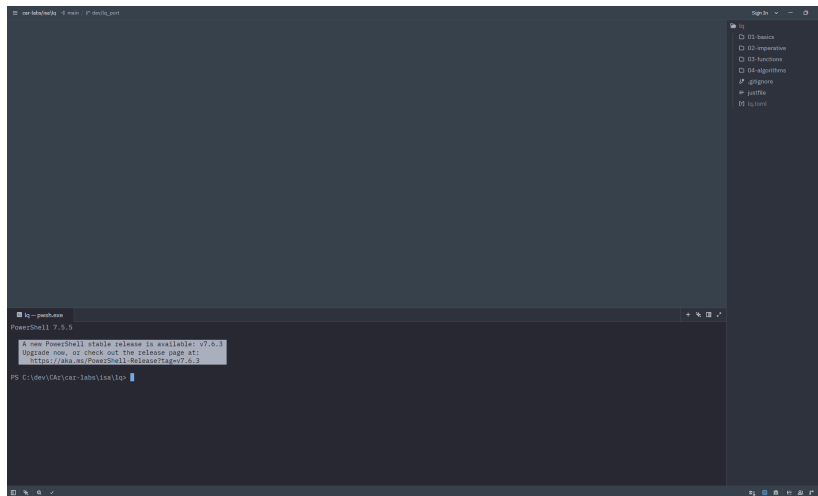


Figure 6 - Open exercises in Zed

Öffnen Sie das Terminal über **View > Terminal Panel**, geben Sie **lq** ein und drücken Sie **Enter**:

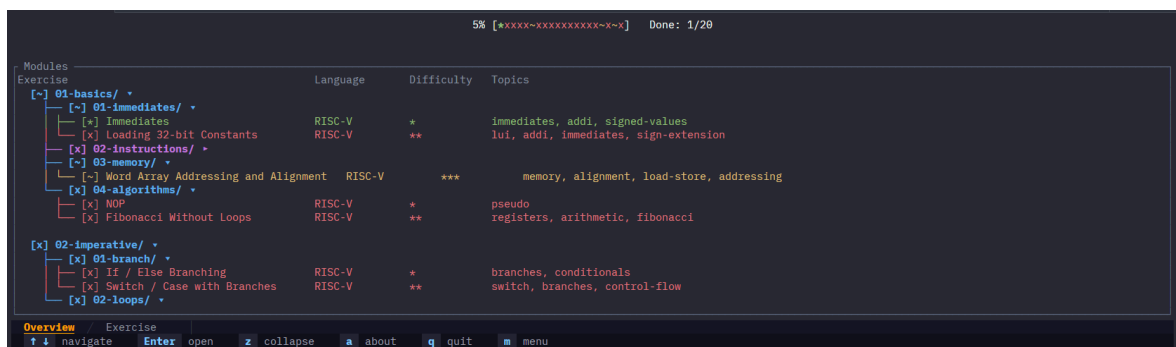
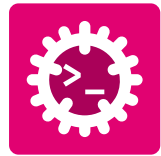


Figure 7 - **lq** in Zed



Wenn das Programm nicht startet, lesen Sie die Terminal-Ausgabe. Die meisten Fehler entstehen, wenn **lq** ausserhalb eines Übungsordners gestartet wird oder das Layout ungültig ist.



Die Programmoberfläche sieht wie folgt aus:

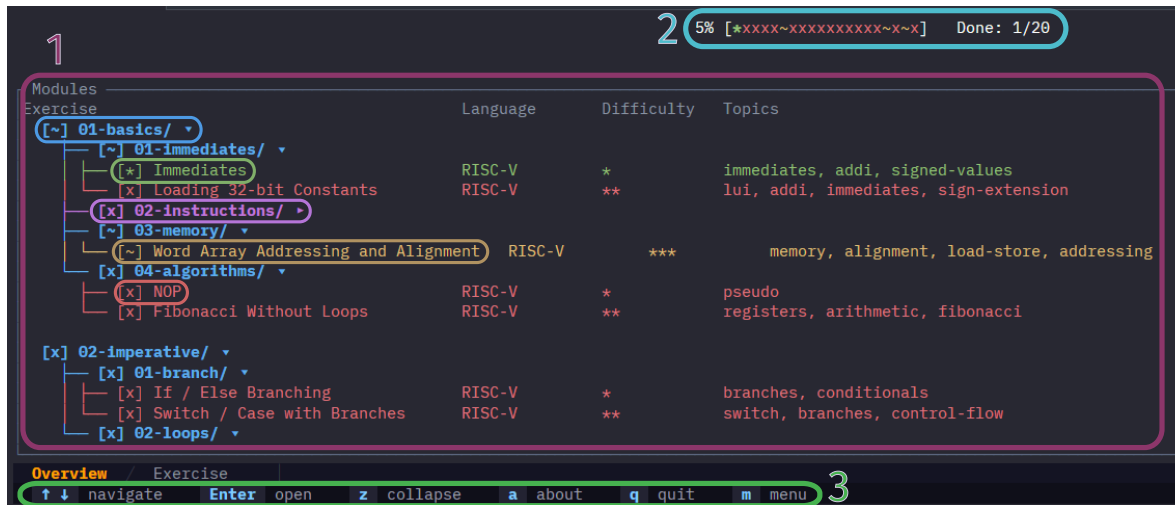


Figure 8 - lq overview page

1. Übungsliste

Gefundene Übungen werden geordnet angezeigt. Navigieren Sie mit `↓` `↑` und wählen Sie mit **Enter**.

- **Blaue** Zeilen markieren Module/Gruppen von Übungen. Mit ***z*** lassen sich Module ein- und ausklappen.
- Die **violette** Zeile zeigt die Cursor-Position.
- Die **grüne** Zeile markiert vollständig abgeschlossene Übungen.
- Die **gelbe** Zeile markiert teilweise abgeschlossene Übungen.
- Die **rote** Zeile markiert noch nicht abgeschlossene Übungen.

Für jede Übung zeigt die Ansicht Sprache (Rust, Go, C++, Python, RISC-V-Assembler und Markdown), relativen Schwierigkeitsgrad innerhalb des Moduls sowie behandelte Themen.

3.2.1 Kurzbefehle im Überblick

- `↓` `↑`: in der Übungsliste navigieren
- **Enter**: ein Modul erweitern / eine Übung auswählen
- ***z***: alle Module in der Liste ein-/ausklappen

2. Fortschritt

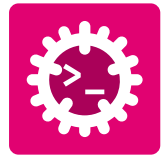
Die Fortschrittsleiste zeigt, wie viele Übungen im Kurs abgeschlossen sind. Sie wird pro Übung wie folgt aktualisiert:

- *****: Übung vollständig abgeschlossen
- **~**: Anforderungen teilweise erfüllt
- **x**: Anforderungen nicht erfüllt

3. Menu

Liste der für die aktuelle Ansicht verfügbaren Tastaturbefehle. Mit ***m*** blenden Sie das Menu ein oder aus.

- ***a***: die About-Seite anzeigen
- ***q*** / ***Esc***: Programm beenden
- ***m***: Menu mit Kurzbefehlen ein-/ausblenden



3.3 Übungsseite

Zwischen den Ansichten wechseln Sie mit $\leftarrow$ $\rightarrow$.

3.3.1 Theorie

Beim Öffnen einer Übung wird der Theorie-Tab angezeigt. Er enthält eine kurze Beschreibung und Erinnerungen an die Kernkonzepte zur Lösung.

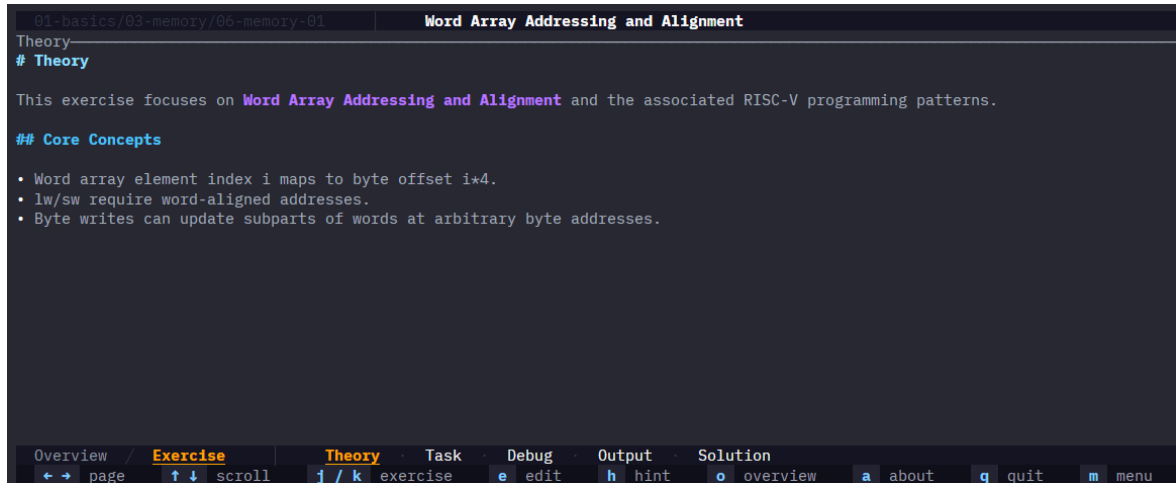


Figure 9 - Theory tab

3.3.2 Aufgaben

Die Aufgabenansicht enthält die Liste der zu erledigenden Aufgaben, Codeausschnitte, Hinweise ...

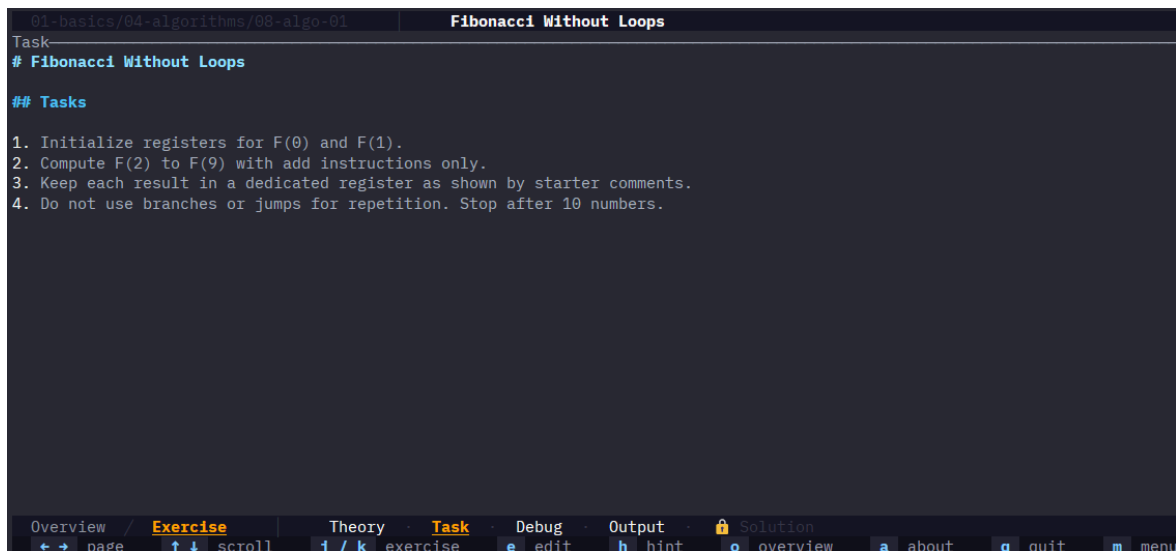
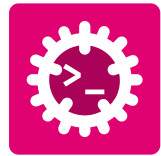


Figure 10 - Tasks tab

3.3.3 Debug

Das Debug-Fenster zeigt die Ausgaben von **stdout** und **stderr** des getesteten Programms. *Dieses Fenster ist nicht nützlich für RISC-V und Markdown.*



```
01-rust/01-hello-world | Hello, Rust!
Debug
stdout: Command Line Output Text to help for debug
stderr: Command Line Output Text to help for debug
```

Overview / **Exercise** | Theory | Task | **Debug** | Output | Solution
 ← → page | ↑ ↓ scroll | j / k exercise | e edit | h hint | o overview | a about | q quit | m menu

Figure 11 - Debug tab

3.3.4 Ausgabe

Das Ausgabefenster zeigt die Testergebnisse. Die Tests sind direkt im Übungscode definiert; schauen Sie dort bei Fehlern nach, um sie besser zu verstehen.

Es zeigt die Gesamtzahl bestandener Tests sowie die fehlgeschlagenen, im jeweils sprachspezifischen Format.

Der Schwellenwert rechts ist der Mindestprozentsatz bestandener Tests, damit die Übung als **teilweise abgeschlossen** oder **vollständig abgeschlossen** gilt.

Dieser Schwellenwert ist vor allem für Markdown-Übungen relevant, da sie anders getestet werden.

```
01-basics/03-memory/06-memory-01 | Word Array Addressing and Alignment
Output [0%]
Score: [=====] 31/32 checks (threshold: 80%)
PASSED ✓
Cycles: 33
✓ x0 (zero) = 0 (0x00000000)
✓ x1 (ra) = 2 (0x00000002)
✓ x2 (sp) = 3 (0x00000003)
✓ x3 (gp) = 4 (0x00000004)
✓ x4 (tp) = 5 (0x00000005)
✓ x5 (t0) = 6 (0x00000006)
✓ x6 (t1) = 7 (0x00000007)
✓ x7 (t2) = 8 (0x00000008)
✓ x8 (s0) = 9 (0x00000009)
✓ x9 (s1) = 10 (0x0000000a)
X x10 (a0): expected 11 (0x0000000b), got 12 (0x0000000c)
✓ x11 (a1) = 12 (0x0000000c)
✓ x12 (a2) = 13 (0x0000000d)
✓ x13 (a3) = 14 (0x0000000e)
✓ x14 (a4) = 15 (0x0000000f)
```

Overview / **Exercise** | Theory | Task | Debug | **Output** | Solution
 ← → page | ↑ ↓ scroll | j / k exercise | e edit | h hint | o overview | a about | q quit | m menu

Figure 12 - Output tab

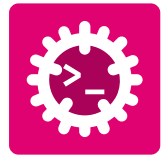
Drucken Sie ***e***, um die Übung im zuvor in Abschnitt 2.2.3 festgelegten Standardeditor zu öffnen. Ist derselbe Editor wie für **lq** gesetzt (z. B. Zed), öffnet sich die Datei in einem neuen Tab im selben Fenster.

3.3.5 Tipps und Lösung

Standardmäßig ist die Lösung gesperrt, wie das Schloss-Symbol zeigt  **Solution**

Sie wird unter zwei Bedingungen freigeschaltet:

- Der Abschlussschwellenwert der Übung ist erreicht.
- Alle Tipps sind freigeschaltet.



Zum Freischalten der Tipps drucken Sie ***h***; dann erscheint folgende Ansicht im Tab **Output**:

```
01-basics/04-algorithms/08-algo-01 | Fibonacci Without Loops
Output
Set the two seed values first: 0 and 1.
[HINT 2/2]
For each next term, add the previous two registers.

⚠ The solution will be unlocked. Are you really sure?
Press 'h' again to confirm, or any other key to cancel.

Overview / Exercise Theory Task Debug Output Solution
← → page ↑ ↓ scroll j / k exercise e edit h hint o overview a about q quit m menu
```

Figure 13 - Output tab

Sobald alle Tipps freigeschaltet sind, entsperrt ein erneutes Drucken von **"h"** die Lösung mit folgender Ansicht:

```
01-basics/02-instructions/01-instruction-01 | Arithmetic Expression (Positive Values)
Solution
— Reference Solution —

# EXPECT_REG: t0 1
# EXPECT_REG: t1 2
# EXPECT_REG: t2 5
# EXPECT_REG: s0 -2

_start:
# a = b + c - d;
# t0 = b, t1 = c, t2 = d. s0 = d

# b = 1, c = 2, d = 5
addi t0, zero, 1
addi t1, zero, 2
addi t2, zero, 5
# t = b + c
add t3, t0, t1
# a = t - c
sub s0, t3, t2

Explanation
## Explanation

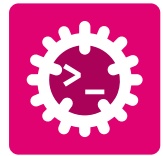
You practice expression lowering into primitive arithmetic instructions with explicit temporaries.
Read solution/main.asm together with the hints, then compare with main.asm to understand each implementation choice.

Overview / Exercise Theory Task Debug Output Solution
← → page ↑ ↓ scroll j / k exercise e edit h hint o overview a about q quit m menu
```

Figure 14 - Solution tab

3.3.6 Kurzbefehle im Überblick

- **← →**: zwischen den Ansichten der Übung wechseln
- **↓ ↑**: in den Ansichten scrollen
- ***j*** / ***k***: zur vorherigen/nachsten Übung wechseln
- ***e***: Übung im Standardeditor öffnen
- ***h***: Tipps anzeigen + Lösung freischalten
- ***o***: zur Übersichtsseite zurück - Abschnitt 3.2
- ***a***: About-Seite anzeigen
- ***q*** / ***Esc***: Programm beenden
- ***m***: Kurzbefehls-Menü ein-/ausblenden



3.4 Info-Seite

Zur Nennung der Personen hinter LangQuest:

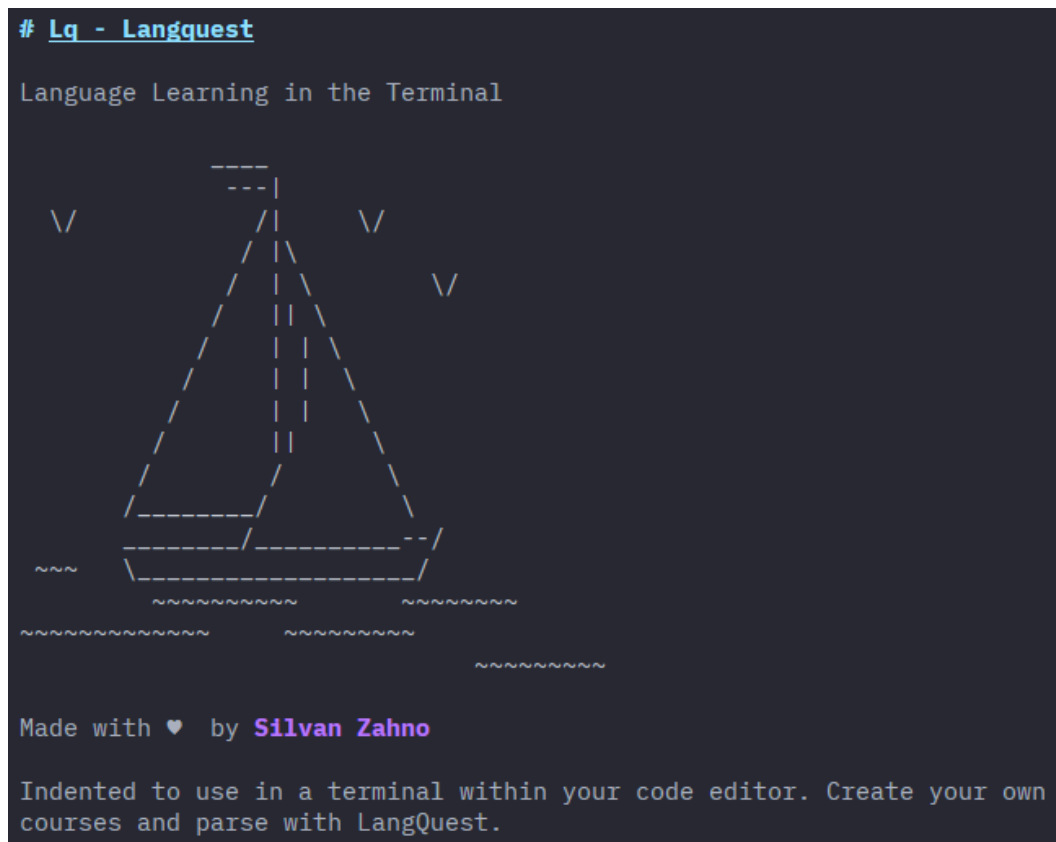
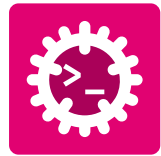


Figure 15 - About page

3.5 Fehlerbehebung

Problem	Ursache	Lösungsschritte
rustup , cargo oder rustc nicht verfügbar	Rust-Toolchain ist nicht korrekt installiert	1. Wiederholen Sie die Rust-Installation über https://rust-lang.org/tools/install/. 2. Starten Sie die Terminal-Sitzung neu. 3. Prüfen Sie mit rustup --version, rustc --version und cargo --version.
Befehl lq nicht gefunden	LangQuest ist nicht installiert oder Cargo-Binaries fehlen im PATH	1. Führen Sie cargo install --git https://github.com/tschinz/langquest aus. 2. Prüfen Sie cargo --version und lq --version. 3. Fügen Sie das Cargo-Bin-Verzeichnis zum PATH hinzu und starten Sie das Terminal neu.



Problem	Ursache	Lösungsschritte
Programm wird gestartet, beendet sich aber sofort	Falsches Arbeitsverzeichnis oder ungültige Übungsstruktur	1. Prüfen Sie die Terminal-Ausgabe. Meistens wird angezeigt, dass keine Übungen gefunden wurden. 2. Wechseln Sie in das Stammverzeichnis mit den Übungsmodulen. 3. Bestätigen Sie, dass die Ordnerstruktur dem erwarteten Muster NN-modul/NN-übung entspricht. 4. Starten Sie lq erneut von diesem Stammverzeichnis aus. 5. Eröffnen Sie ein Issue unter https://github.com/tschinz/langquest/issues.
Taste e öffnet Dateien nicht im erwarteten Editor	Dateityp-Zuordnungen fehlen oder zeigen auf eine andere Anwendung	1. Konfigurieren Sie Standard-Apps für .rs, .py, .go, .cpp, .md usw. neu. 2. Schließen und öffnen Sie den Editor erneut. 3. Starten Sie lq neu.
Lösung bleibt gesperrt	Abschlusschwelle nicht erreicht oder Tipps nicht vollständig freigeschaltet	1. Fixieren Sie Ihren Code, um die Tests zu bestehen, bis die Schwelle erreicht ist. 2. Drücken Sie h, bis alle Tipps freigeschaltet sind. 3. Drücken Sie h erneut, um den Lösungstab freizuschalten.
Programmoberfläche hängt oder reagiert nicht mehr	Derzeit arbeitet lq nur mit einem Thread. Wenn eine Übung geöffnet oder die Übungsdatei gespeichert wird, erfolgt eine Kompilierung + Ausführung der Datei. Wenn eine dieser Aktionen lange dauert, kann die Oberfläche hängen. Zum Beispiel kann eine Endlosschleife im Übungscode zu einem Hängenbleiben führen.	1. Prüfen Sie, ob der Übungscode keine Endlosschleife enthält. 2. Versuchen Sie eine Kompilierung außerhalb von lq. 3. Prüfen Sie, ob eine neuere Version von LangQuest verfügbar ist.



Glossary

AI – Artificial Intelligence: Field of computer science focused on creating systems capable of performing tasks that typically require human intelligence. [9](#)